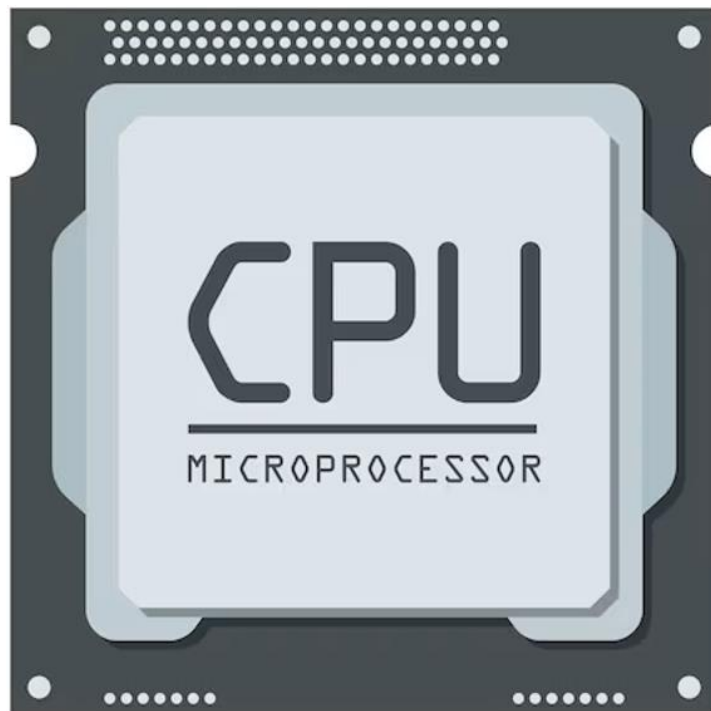


EL KATTOUFI Youssef

Année 2025 – 2026

Processeur Monocycle et Multicycle



Etablissement scolaire : Polytech Sorbonne

Formation : Electronique et Informatique

Matières : VHDL comportemental

Enseignant : Yan DOUZE

Sommaire

Remerciements.....	4
Résumé	5
I. Présentation du projet	6
II. Conception et Architecture Fonctionnelle	7
A. Unité Arithmétique et Logique (ALU).....	8
B. Bascule D active sur front montant	Erreur ! Signet non défini.
C. Registre à changement parallèle (4bits)	Erreur ! Signet non défini.
D. Stockage du code	Erreur ! Signet non défini.
E. Affichage 7 segments.....	Erreur ! Signet non défini.
F. Comparateur 4bits	Erreur ! Signet non défini.
G. Comparateur 16 bits	Erreur ! Signet non défini.
III. Assemblage : Serrure électronique complète	Erreur ! Signet non défini.
A. Fonctionnement top module	Erreur ! Signet non défini.
B. Fonctionnement test bench	Erreur ! Signet non défini.
IV. Quartus prime	26
Sources	27
Tables des figures	28

Remerciements

Je tiens tout d'abord à remercier chaleureusement Monsieur Yan Douze qui nous a encadré tout au long du semestre 6. Son accompagnement, ses conseils et son enseignement m'a permis de mener à bien ce projet, tout en nous apportant le socle de connaissances nécessaires en vhdl comportemental.

Résumé

Au sein de la formation d'ingénieur en Électronique et Informatique à Polytech Sorbonne, nous avons suivi l'enseignement de VHDL comportemental, qui nous a permis d'acquérir des connaissances fondamentales sur l'architecture et le fonctionnement des processeurs monocycle et multicycle.

Cette matière nous a notamment initiés aux différents composants internes d'un processeur, tels que l'ALU, le décodeur d'instructions, l'unité de contrôle, la mémoire de données et les registres. Nous avons ainsi pu mieux comprendre les mécanismes qui permettent l'exécution des instructions et la gestion des flux de données au sein d'un système numérique.

Cet enseignement m'a également sensibilisé à la complexité de la conception et de l'implémentation des processeurs. J'ai notamment pris conscience de l'importance des phases de simulation et de validation du système avant sa synthèse sur FPGA. En effet, des simulations bien faites permettent de détecter les erreurs en amont, de réduire le temps de développement et d'éviter de longues phases de compilation sous Quartus Prime.

Enfin, cette expérience a mis en évidence l'importance de la modélisation et de l'anticipation du comportement d'un système numérique, afin de vérifier son bon fonctionnement avant toute réalisation matérielle.

I. Présentation du projet

Ce mini-projet a pour objectif la conception, la simulation et l'implémentation sur FPGA d'un processeur inspiré de l'architecture ARM7TDMI, en langage VHDL sur la carte DE10-Lite (voir figure ci-dessous). Le but est de reproduire le fonctionnement interne d'un processeur capable d'exécuter un jeu d'instructions de type ARM, en mettant en œuvre les principaux blocs d'un calculateur numérique.

Deux architectures ont été étudiées : le processeur monocycle, où chaque instruction est exécutée en un seul cycle d'horloge, et le processeur multicycle, où l'exécution est répartie sur plusieurs cycles afin d'optimiser l'utilisation des ressources matérielles.

À travers ce travail, nous avons pu analyser le chemin de données et le chemin de contrôle du processeur, comprendre la gestion des instructions, des accès mémoire et des opérations arithmétiques, puis valider chaque module par simulation logique avant de l'exporter sur la carte DE10-Lite. Des bancs de test et des scripts de simulation automatisés sous ModelSim ont été développés afin de vérifier que le comportement était le bon.

Une fois la conception validée, le processeur a été synthétisé et déployé sur la carte FPGA à l'aide de l'environnement Intel Quartus Prime, incluant l'assignation des broches.

Il met également en évidence l'importance des phases de simulation et de vérification en amont, qui permettent de détecter les erreurs dès la conception, de réduire le temps de développement et d'éviter des pertes de temps lors de l'implémentation sur FPGA.

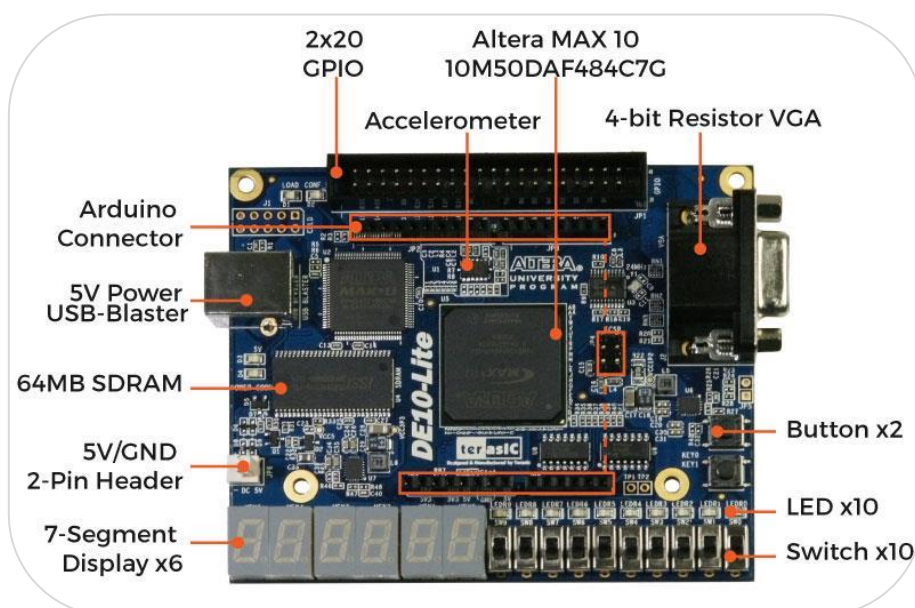


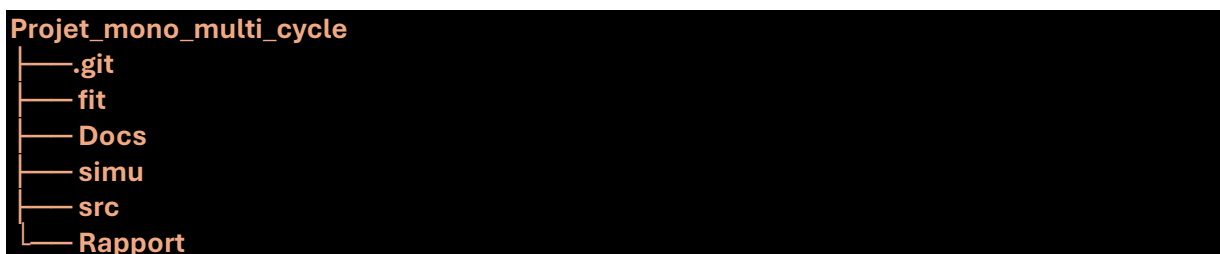
Figure 1 : Carte DE10-Lite

II. Conception et Architecture Fonctionnelle

Dans ce compte-rendu, nous présentons la démarche complète de conception des processeurs monocycle et multicycle. Nous détaillons les choix d'architecture, les schémas de conception, le code VHDL ainsi que les simulations associées à chaque module, afin de permettre une compréhension progressive du fonctionnement global du système et la manière dont moi je l'ai écrits.

- ✚ Unité Arithmétique et Logique (ALU);
- ✚ Banc de registre ;
- ✚ Datapath ;
- ✚ Multiplexeur 2 vers 1 ;
- ✚ Extension de Signe ;
- ✚ Assemblage de l'Unité de Traitement ;
- ✚ Unité de Gestion des Instruction ;
- ✚ Registre 32 bis avec commande de chargement ;
- ✚ Décodeur d'instructions ;
- ✚ Assemblage du processeur monocycle ;

La méthode de projet adoptée a consisté à organiser clairement les fichiers du projet selon une architecture proposée par Monsieur Yan Douze afin d'éviter toute confusion. Cette organisation s'est avérée plus efficace que celle que j'avais mise en place au premier semestre et a facilité le développement, la gestion des fichiers ainsi que les phases de simulation.



Le dossier simu regroupe l'ensemble des bancs de test ainsi que les scripts de simulation permettant de valider le comportement des différents modules/composants VHDL. Le dossier fit contient les fichiers nécessaires à la synthèse et au déploiement du projet sur la carte DE10-Lite à l'aide du logiciel Quartus Prime. Le dossier src rassemble l'ensemble des fichiers VHDL

décrivant les différents composants du processeur, tandis que le dossier Rapport contient les documents associés à la rédaction du compte-rendu.

Par ailleurs, afin d'assurer un suivi du développement et de conserver un historique des modifications, un dépôt GitHub a été utilisé pour l'archivage du projet et ainsi pouvoir revenir en arrière en cas de problème.

Pour chacune de ces parties, il existe un fichier `simu.do` permettant d'automatiser les bancs de tests.

A. Unité Arithmétique et Logique (ALU)

L'Unité Arithmétique et Logique (ALU) développée est un circuit combinatoire 32 bits. Elle réalise des opérations arithmétiques ou des transferts directs de données en fonction d'un code opératoire, tout en mettant à jour des indicateurs d'état.

1. *Explication du code de l'ALU*

Le fonctionnement de l'ALU est contrôlé par un processus combinatoire sensible aux entrées A, B et OP dont le fichier est **`alu.vhd`**. Le signal OP sur 2 bits permet de sélectionner quatre comportements différents :

- ✚ "00" : addition ($S = A + B$). Les signaux sont convertis en unsigned pour effectuer l'opération arithmétique, puis réassignés sous forme de vecteurs de bits.
- ✚ "01" : passage de B ($S = B$). La sortie recopie directement l'opérande B.
- ✚ "10" : soustraction ($S = A - B$). L'opération est réalisée en logique non signée via le type unsigned.
- ✚ "11" : passage de A ($S = A$). La sortie recopie directement l'opérande A.

Entity: ALU

- File: `alu.vhd`

Diagram

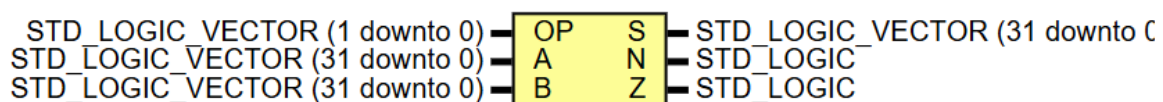


Figure 2 : Représentation graphique de l'ALU

Les drapeaux sont calculés en continu à partir du signal interne `result_internal` :

- ✚ Drapeau négatif (N) : il correspond au bit de poids fort du résultat (result_internal(31)). Si ce bit vaut '1', le résultat est négatif en complément à deux.
- ✚ Drapeau zéro (Z) : il passe à '1' si et seulement si les 32 bits du résultat sont tous à '0'.

2. Explication du banc de test de l'ALU

Le fichier **tb_alu.vhd** permet de vérifier le comportement de l'ALU sans entrées physiques, grâce à un scénario de simulation temporel.

Le composant ALU est instancié dans le testbench. Ses ports (OP, A, B, S, N et Z) sont connectés à des signaux locaux initialisés à zéro afin de garantir un état de départ connu.

Le processus de stimulation applique successivement plusieurs cas de test, séparés par un délai de 10 ns (wait for 10 ns) pour permettre la stabilisation des signaux.

Les tests réalisés sont les suivants : une addition ($10 + 5$), le transfert de l'opérande B, une soustraction ($10 - 5$), la vérification du flag Z avec l'opération $10 - 10 = 0$, la vérification du flag N avec l'opération $5 - 10 = -5$, puis le transfert de l'opérande A à l'aide de la valeur hexadécimale X"ABCDEFFF". Ces essais permettent de valider les principales fonctionnalités de l'ALU ainsi que le comportement des indicateurs d'état.

3. Résultat de la simulation de l'ALU

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simALU.do**.

La figure ci-dessous issue de ModelSim valide le comportement attendu de l'ALU à chaque étape de 10 ns. Les signaux A, B et S sont affichés en hexadécimal pour faciliter la lecture.

- ✚ 0 à 10 ns (addition) : OP = "00". A = 10 et B = 5. La sortie S vaut 15. Les flags N et Z restent à 0.
- ✚ 10 à 20 ns (passage de B) : OP = "01". S recopie B et vaut 5.
- ✚ 20 à 30 ns (soustraction) : OP = "10". Le calcul donne $10 - 5 = 5$.
- ✚ 30 à 40 ns (flag Z) : A = 10 et B = 10. Le résultat est 0, donc Z passe à 1.
- ✚ 40 à 50 ns (flag N) : A = 5 et B = 10. Le résultat est négatif (-5), donc N passe à 1 et Z revient à 0.
- ✚ 50 à 60 ns (passage de A) : OP = "11". A = X"ABCDEFFF". La sortie S recopie cette valeur. Le bit de poids fort étant à 1, le flag N reste actif.

Entrees ALU	Msgs								
/tb_alu/OP	-No Data-	2						3	
/tb_alu/A	-No Data-	0000000A				00000005		ABCDEFFF	
/tb_alu/B	-No Data-	00000005		0000000A					
Resultats									
/tb_alu/S	-No Data-	00000005		00000000		FFFFFFF8		ABCDEFFF	
Flags									
/tb_alu/N	-No Data-								
/tb_alu/Z	-No Data-								

Figure 3 : Simulation ALU

B. Banc de registre

1. Explication du code du banc de registre

À la mise sous tension ou lors d'un signal de reset, tous les registres de R0 à R14 sont mis à zéro. En revanche, le registre R15 (correspondant au compteur de programme ou PC) est initialisé avec la valeur 0x00000030 (48 en décimal), conformément aux directives du projet.

L'accès en lecture est purement combinatoire. Dès que les adresses RA ou RB changent, les données des registres correspondants sont immédiatement disponibles sur les bus de sortie A et B. Cela permet d'alimenter directement l'ALU sans attendre de front d'horloge.

L'écriture d'une donnée sur le bus W à l'adresse RW est séquentielle. Elle nécessite un front montant d'horloge (rising_edge(Clk)) ainsi que l'activation du signal d'autorisation d'écriture (WE = '1').

Entity: BANC_REGISTRE

• File: banc_registre.vhd

Diagram

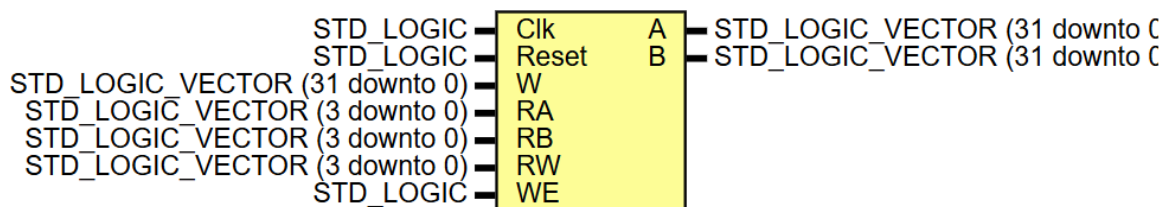


Figure 4 : Représentation graphique du Banc de registre

2. Explication du banc de test du banc de registre

Le banc de test fourni est **tb_datapath.vhd** (sans le V2), il instancie l'unité globale DataPath. Cependant, l'objectif de cette simulation est de valider uniquement le couplage entre le banc de registres et l'ALU. Par la même occasion montrée que le banc de registre fonctionne.

Remarque :

Bien que le fichier **datapath.vhd** contienne d'autres modules (comme la mémoire de données, l'extension de signe ou les multiplexeurs), ceux-ci ont été volontairement désactivés. L'intégration de ces éléments sera présentée plus tard dans le rapport.

Le scénario de test enchaîne cinq opérations arithmétiques consécutives, synchronisées par une horloge de période 10 ns :

- ✚ Copie du contenu de R15 (0x30) vers le registre R1 ;
- ✚ Première accumulation : somme $R1 + R15$ ($0x30 + 0x30$), résultat 0x60 stocké dans R1 ;
- ✚ Seconde accumulation : somme $R1 + R15$ ($0x60 + 0x30$), résultat 0x90 stocké dans R2 ;
- ✚ Première soustraction : $R1 - R15$ ($0x60 - 0x30$), résultat 0x30 stocké dans R3 ;
- ✚ Seconde soustraction (négative) : $R7 - R15$ ($0 - 0x30$), résultat -0x30, soit X"FFFFFFD0" en complément à deux.

3. *Résultat de la simulation du banc de registre*

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simDATAPATH.do (sans le V2)**.

La figure ci-dessous issue de ModelSim valide le comportement attendu du banc de registre et par la même occasion le datapath. Les valeurs des bus et des registres sont ici représentées au format hexadécimal.

- ✚ De 0 à 20 ns (Phase de Reset) : Le signal "rst" est actif à '1'. Le système s'initialise.
- ✚ De 20 à 30 ns (Stabilisation) : Le signal "rst" repasse à '0'. Le "BusB" affiche déjà 00000030 car l'adresse de lecture "RB" pointe sur X"F" (15) et la lecture est asynchrone.
- ✚ De 30 à 40 ns (Test 1 - Copie de "R15" dans "R1") : "RB" vaut F ("R15") et l'ALU est configurée en transfert de B ("aluctr" = 01). La sortie "Aluout" prend la valeur 00000030. Comme "MemtoReg" vaut 0, cette valeur est transmise sur "BusW". Au front montant d'horloge à 35 ns, elle est écrite dans "R1" ("RW" = 1).
- ✚ De 40 à 50 ns (Test 2 - " $R1 = R1 + R15$ ") : "RA" = 1 et "RB" = F. "BusA" et "BusB" affichent tous deux 00000030. L'ALU réalise une addition ("Aluotr" = 00) et produit 00000060 sur "Aluout" et "BusW". Cette valeur est enregistrée dans "R1" au front montant de 45 ns.
- ✚ De 50 à 60 ns (Test 3 - " $R2 = R1 + R15$ ") : "RA" reste à 1 (valeur 00000060) et "RB" = F (00000030). L'ALU calcule 00000090, présent sur "Aluout" et "BusW". Le résultat est écrit dans "R2" au front montant de 55 ns.

- De 60 à 70 ns (Test 4 - "R3" = "R1" - "R15") : L'ALU effectue une soustraction ("Alucontr" = 10) entre 00000060 et 00000030. Le résultat 00000030 est placé sur "Aluout" et écrit dans "R3" au front montant de 65 ns.
- De 70 à 80 ns (Test 5 - "R5" = "R7" - "R15") : "RA" = 7, donc "BusA" = 00000000, tandis que "BusB" = 00000030. L'ALU calcule 0 - 0x30, donnant FFFFFFFD0 en complément à deux. Cette valeur est transmise sur "BusW" et enregistrée dans "R5" au front montant de 75 ns. Le flag négatif "N_flag" passe à '1', confirmant le résultat négatif.

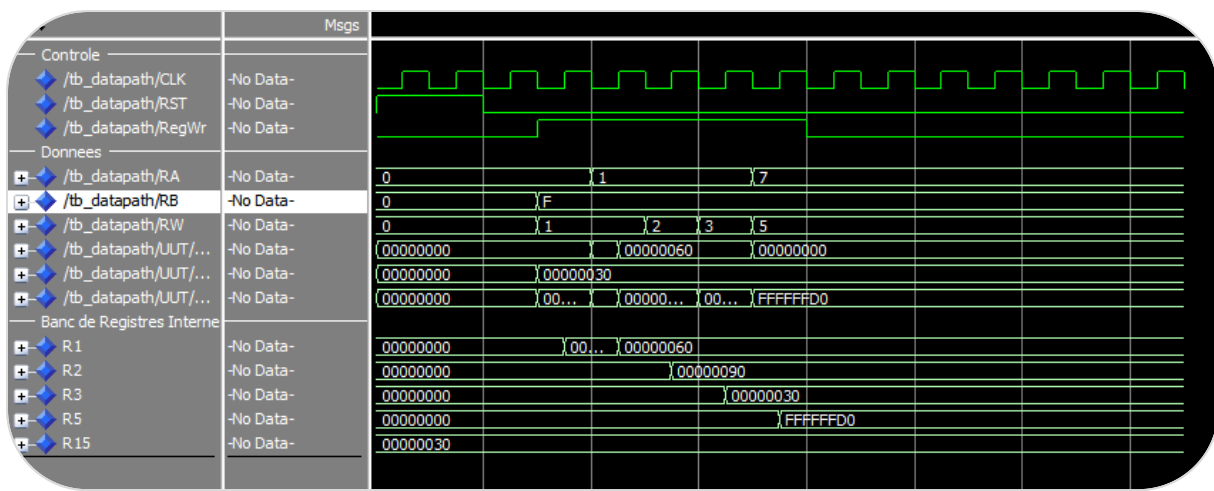


Figure 5 : Simulation Datapath (Banc de registre + ALU)

C. Le multiplexeur 2 vers 1

1. Explication du code du multiplexeur 2 vers 1

Le composant intègre un paramètre générique N_bits (fixé par défaut à 32). Cela permet de le réutiliser à différents endroits du Datapath avec des tailles de bus différentes, sans avoir à réécrire le code.

L'architecture utilise une affectation conditionnelle directe (when... else). Le signal de commande COM détermine quelle entrée est envoyée vers la sortie S :

- Si COM = '0', la sortie S recopie le bus d'entrée A ;
- Si COM = '1', la sortie S recopie le bus d'entrée B ;

Entity: MUL_2_TO_1

- File: mul_2_to_1.vhd

Diagram

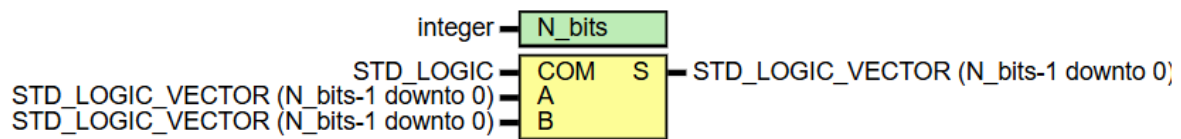


Figure 6 : Représentation graphique du Multiplexeur 2 vers 1

2. Explication du banc de test du multiplexeur 2 vers 1

Le fichier **tb_mul_2_to_1.vhd** permet de vérifier le comportement de du multiplexeur 2 vers 1.

Le banc de test instancie le multiplexeur en fixant la valeur de N_bits à 32 afin de simuler l'aiguillage de mots de données complets.

Le processus applique une série de stimuli temporels espacés de 10 ns pour valider les différentes configurations du multiplexeur :

- ✚ Initialisation (COM = 0) : application de deux motifs de bits alternés (A = 0xAAAAAAAA et B = 0x55555555) ;
- ✚ Bascule (COM = 1) : maintien des mêmes entrées afin d'observer le changement de la sortie ;
- ✚ Nouvelles valeurs (COM = 0) : modification simultanée des entrées A et B avec de nouvelles valeurs (0x12345678 et 0x87654321) ;
- ✚ Dernière bascule (COM = 1) : changement final de la commande afin de vérifier la transmission correcte de la valeur de B.

3. Résultat de la simulation du multiplexeur 2 vers 1

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simuMUL_2_TO_1.do**.

La figure ci-dessous issue de ModelSim valide le comportement attendu du multiplexeur. Les valeurs des bus A, B et S sont affichées en notation hexadécimale et confirment le fonctionnement combinatoire et instantané du composant.

- ✚ De 0 à 10 ns (test 1) : la commande com est positionnée à '0'. L'entrée A vaut AAAAAAAAAA et B vaut 55555555. Fidèle à sa table de vérité, le multiplexeur sélectionne l'entrée A : la sortie S prend immédiatement la valeur AAAAAAAAAA.
- ✚ De 10 à 20 ns (test 2) : les entrées A et B restent inchangées, mais le signal de commande com passe à '1'. Le multiplexeur bascule instantanément : la sortie S ne copie plus A mais B, prenant la valeur 55555555.
- ✚ De 20 à 30 ns (test 3) : le signal de commande com repasse à '0' pendant que les entrées reçoivent de nouvelles valeurs : A prend 12345678 et B prend 87654321. La commande étant à '0', c'est le nouveau contenu de A qui est sélectionné : la sortie S affiche 12345678.
- ✚ De 30 à 40 ns (test 4) : enfin, la commande com repasse à '1'. Le circuit combinatoire réagit sans aucun retard d'horloge et propage la nouvelle valeur de B : la sortie S se stabilise à 87654321.

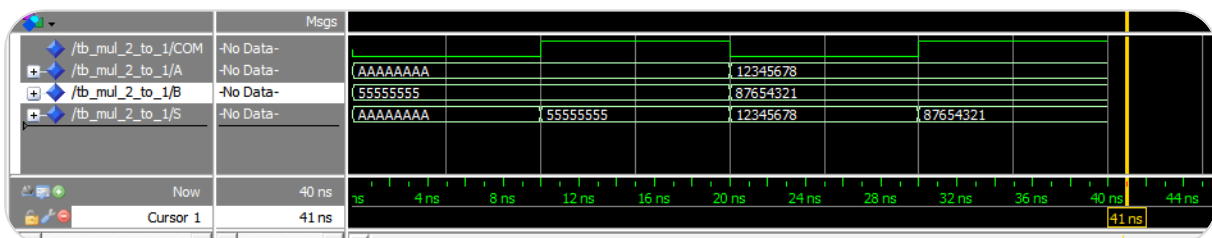


Figure 7 : Simulation du Multiplexeur 2 vers 1

D. Extension de signe

1. Explication du code de l'extension du signe

Le module d'extension de signe (SIGN_EXTENSION) est un circuit combinatoire utilisé pour transformer une valeur immédiate de taille réduite en un format 32 bits compatible avec le fonctionnement interne de l'ALU.

La généricité du module, via le paramètre « N_bits_E », permet d'adapter facilement sa taille d'entrée. Dans notre cas, l'entrée est configurée sur 8 bits, mais le module peut également être utilisé pour d'autres architectures nécessitant des largeurs différentes, par exemple 12 ou 24 bits dans certains formats d'instructions de type ARM.

Le principe de l'extension de signe repose sur la conservation de la valeur en complément à deux lors du passage à une taille supérieure. Pour cela, le bit de poids fort de l'entrée (MSB), qui représente le signe, est recopié sur l'ensemble des bits ajoutés. Cela garantit que la valeur reste inchangée, qu'elle soit positive ou négative.

L'architecture repose sur un processus combinatoire sensible à "E". Les bits de poids faible de la sortie "S" (de 0 à N_bits_E - 1) recopient directement les bits de l'entrée "E". Les bits restants (de N_bits_E à 31) sont remplis avec la valeur du bit de signe de "E" grâce à l'expression (others => E(N_bits_E - 1)).

Entity: SIGN_EXTENSION

- File: sign_extension.vhd

Diagram

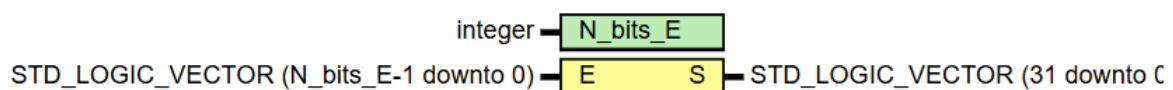


Figure 8 : Représentation graphique de l'Extension de signe

2. Explication du banc de test du multiplexeur 2 vers 1

Le fichier **tb_sign_extension.vhd** permet de vérifier le comportement du composant.

Le banc de test configure le paramètre « N_bits_E » à 8 afin de simuler l'extension d'un octet signé vers un mot de 32 bits.

Le processus applique quatre profils de bits différents toutes les 10 ns :

- ✚ Valeur positive (00000101) : équivaut à +5 en décimal. Le bit de signe (MSB) vaut '0' ;
- ✚ Valeur négative (11111011) : équivaut à -5 en décimal. Le bit de signe (MSB) vaut '1' ;
- ✚ Autre valeur négative (10000001) : équivaut à -127 en décimal. Le bit de signe (MSB) vaut '1' ;
- ✚ Zéro (00000000) : cas limite, le bit de signe vaut '0'.

3. Résultat de la simulation du multiplexeur 2 vers 1

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simu**
simuSIGN_EXTENSION.do.

La figure ci-dessous issue de ModelSim montre le bon fonctionnement de l'extension de signe. L'entrée **e** est affichée en binaire afin d'observer son bit de signe, tandis que la sortie **s** sur 32 bits est affichée en notation binaire.

- ✚ De 0 à 10 ns (test 1 - entrée positive) : l'entrée **e** reçoit "00000101". Le bit de signe vaut '0'. Le module propage donc des '0' sur les bits supérieurs de la sortie. On observe sur le bus **s** la valeur 00000005. La valeur +5 est conservée.
- ✚ De 10 à 20 ns (test 2 - entrée négative) : l'entrée **e** passe à "11111011". Le bit de signe vaut '1'. Le module l'identifie et remplit les bits supérieurs de la sortie avec des '1'. En hexadécimal, ces bits forment la valeur FFFFFFFB, ce qui correspond à -5 en complément à deux sur 32 bits.
- ✚ De 20 à 30 ns (test 3 - seconde entrée négative) : l'entrée **e** prend la valeur "10000001". Le bit de poids fort étant toujours à '1', l'extension applique la même règle de propagation des '1'. La sortie **s** devient FFFFFFF8, en conservant la valeur négative.
- ✚ De 30 à 40 ns (test 4 - zéro) : l'entrée **e** vaut "00000000". Le bit de signe est à '0', donc la sortie est complétée avec des zéros. La sortie **s** affiche 00000000.

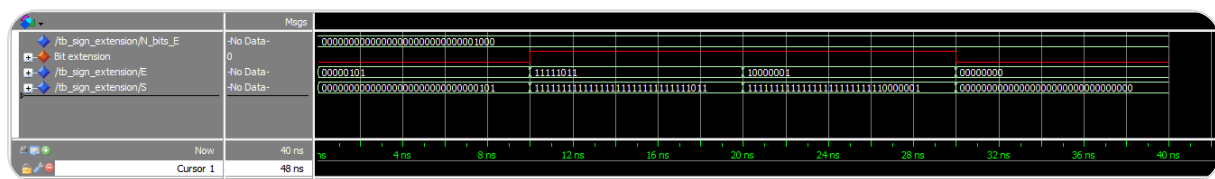


Figure 9 : Simulation de l'Extension du signe

E. Mémoire de données

1. Explication du code sur la mémoire de données

Le module DATA_MEMORY modélise une mémoire vive (RAM) de type synchrone en écriture et asynchrone en lecture, structurée pour stocker 64 mots de 32 bits.

La mémoire est définie sous la forme d'un tableau de 64 éléments (0 to 63), chacun codé sur 32 bits.

À l'état initial ou lors de l'activation du signal reset, la mémoire charge un profil de données spécifique sur les adresses allant de 16 (0x10) à 26 (0x1A), tandis que le reste des cases est initialisé à zéro. Ce pré-chargement simule la présence de valeurs initiales nécessaires à l'exécution des programmes (comme des données de test ou des zones de stockage pour l'instruction STR).

L'écriture de la donnée DataIn à l'adresse Addr nécessite la validation du signal d'écriture (WrEn = '1') et s'effectue au front montant de l'horloge (rising_edge(CLK)).

Contrairement à l'écriture, la sortie DataOut est directement connectée à la case mémoire pointée par Addr. Tout changement d'adresse se traduit immédiatement par la mise à jour de la donnée en sortie, ce qui permet de réduire le temps d'accès lors des cycles de lecture (instructions de type LDR).

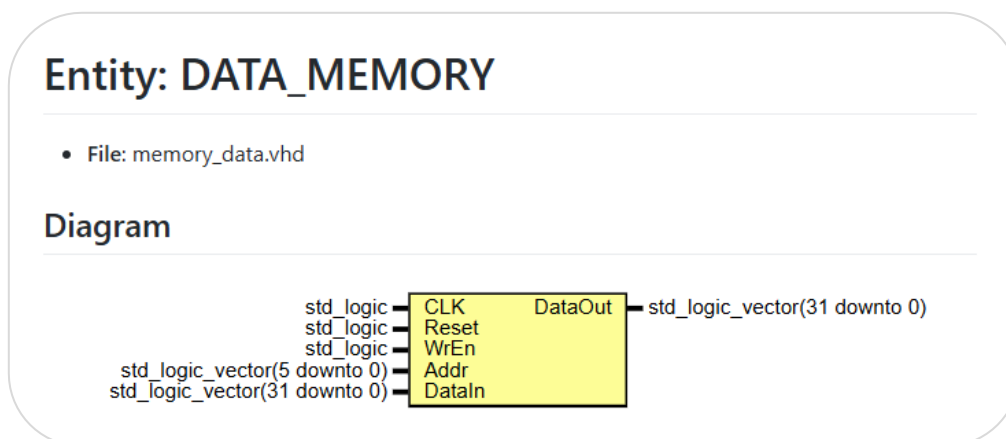


Figure 10 : Représentation graphique de la Memory data

2. Explication du banc de test sur la mémoire de données

Le fichier **tb_memory_data.vhd** permet de vérifier le comportement du composant.

Le banc de test génère une horloge de période 10 ns

- ✚ Phase de reset : activation du signal Reset pendant un cycle complet afin d'initialiser le tableau avec les valeurs par défaut ;
- ✚ Ecriture à l'adresse 0 : configuration de l'adresse 000000, injection du motif de test 0xAAAAAAAA et activation du signal WrEn ;
- ✚ Maintien et lecture : désactivation de WrEn afin de vérifier que la donnée reste stable en mémoire ;
- ✚ Ecriture à l'adresse 10 (0x0A) : configuration de l'adresse 001010, injection de la valeur 0x12345678 et activation de WrEn.

3. *Résultat de la simulation de la mémoire de donnée*

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme ***simu*** ***simuMEMORY_DATA.do***.

La figure ci-dessous issue de ModelSim montre le bon fonctionnement de la mémoire de donnée.

- ✚ De 0 à 10 ns (phase de reset) : le signal reset est actif à '1'. Bien que l'adresse addr pointe sur 00, la sortie dataout affiche 00000000 car la case mémoire 0 vient d'être initialisée à zéro par le bloc de reset.
- ✚ De 10 à 20 ns (écriture à l'adresse 0) : le reset retombe à '0'. L'adresse addr reste à 00, datain présente la valeur AAAAAAAAA et wren passe à '1'. Au front montant de l'horloge situé à 15 ns, l'écriture est validée en interne. Par conséquent, la lecture asynchrone recopie immédiatement cette modification : la sortie dataout bascule à AAAAAAAAA.
- ✚ De 20 à 30 ns (maintien de l'adresse 0) : le signal wren est remis à '0'. La sortie dataout maintient sa valeur AAAAAAAAA, ce qui montre que la donnée a bien été stockée dans la case 0 de la mémoire et qu'aucune écriture supplémentaire n'a lieu.
- ✚ De 30 à 40 ns (écriture à l'adresse 10 / 0x0A) : l'adresse addr bascule à 0A (10 en décimal). Avant même le front d'horloge, on observe que dataout passe instantanément à 00000000 (lecture asynchrone de la case 10 initialement vide). Le signal datain présente 12345678 et wren remonte à '1'. Au front montant de l'horloge à 35 ns, la donnée est enregistrée en mémoire. La sortie dataout affiche alors instantanément le nouveau contenu de la case : 12345678.
- ✚ Après 40 ns (fin de simulation) : le signal wren retombe à '0', ce qui stabilise le système et valide la réussite de l'ensemble des opérations d'accès à la mémoire.

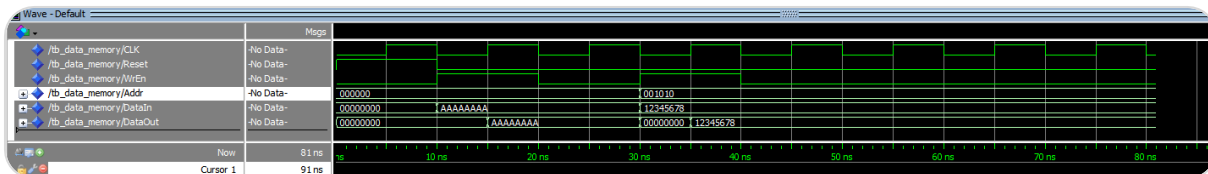


Figure 11 : Simulation de la Memory Data

F. Assemblage Unité de Traitement

Après avoir validé individuellement chaque bloc périphérique (multiplexeurs, extension de signe, mémoire de données) et l'interaction de base entre le banc de registres et l'ALU (avec la première version du datapath), cette étape marque la complétion d'une partie du processeur monocycle.

Entity: DataPath

• File: datapath.vhd

Diagram

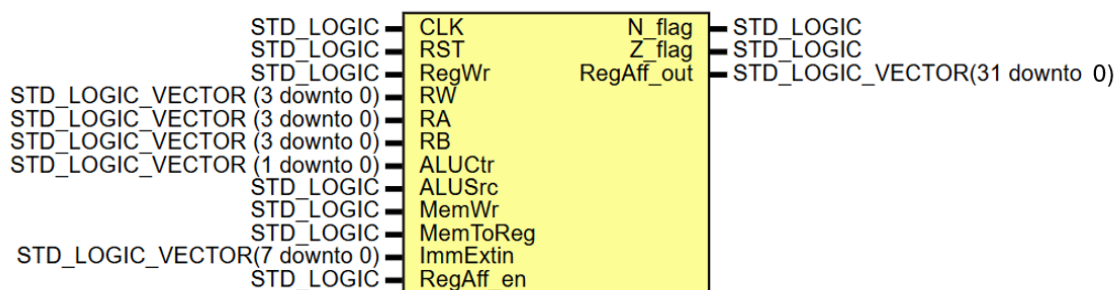


Figure 12 : Représentation graphique du Datapath

Voici l'assemblage effectuer avec les différents composants fait précédement.

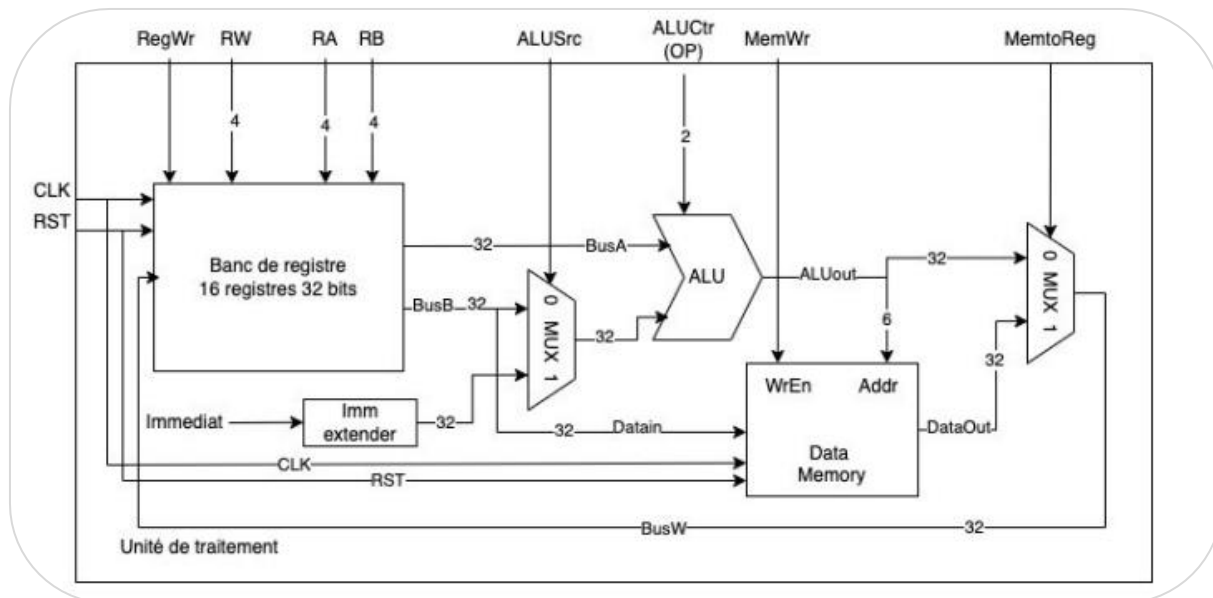


Figure 13 : Assemblage des modules créer précédement

1. Explication du banc de test du datapath

Le fichier **tb_datapathV2.vhd** permet de vérifier le comportement du datapath.

Les différents tests réalisés sont les suivants :

- ✚ Phase d'initialisation : chargement des valeurs de départ avec $R1 = 10$ (0x0A) et $R2 = 20$ (0x14).
- ✚ Test 1 : addition registre-registre : calcul de $R3 = R1 + R2$, soit $10 + 20 = 30$ (0x1E).
- ✚ Test 2 : addition registre-immédiat : calcul de $R4 = R1 + 5$, soit 15 (0x0F).
- ✚ Test 3 : soustraction registre-registre : calcul de $R5 = R2 - R1$, soit $20 - 10 = 10$ (0x0A).
- ✚ Test 4 : soustraction registre-immédiat : calcul de $R6 = R2 - 3$, soit 17 (0x11).
- ✚ Test 5 : copie d'un registre : transfert de la valeur de $R1$ dans $R7$ en réalisant l'opération $R1 + 0$.
- ✚ Test 6 : écriture en mémoire (STR) : stockage de la valeur de $R2$ (20) à l'adresse contenue dans $R1$ (10).
- ✚ Test 7 : lecture mémoire (LDR) : récupération de la valeur stockée à l'adresse 10 et écriture du résultat dans $R8$.

2. Résultat de la simulation du datapath

Le script permettant de mettre en forme l'affichage et de lancer la simulation se nomme **simu simuDATAPATHV2.do**.

La figure ci-dessous issue de ModelSim montre le bon fonctionnement du datapath.



Figure 14 : Simulation du Datapaht

Dans la partie 4.1 du TP, il nous a été demandé de concevoir un détecteur de front montant. Dans un premier temps, cette implémentation fonctionnait correctement en simulation sous ModelSim. Cependant, lors des tests pratiques sur la carte DE10-Lite, un comportement inattendu a été observé : le système ne validait pas la saisie lors de l'appui sur le bouton poussoir, mais uniquement lors de son relâchement.

Cette observation s'explique par le fait que le bouton poussoir de la carte est actif à l'état bas (pull-down). Ainsi, le signal passe de '1' à '0' lors de l'appui, ce qui correspond à un front descendant. Afin d'obtenir un comportement cohérent entre la simulation et le fonctionnement réel sur la carte, le code du détecteur de front ainsi que le banc de test ont été modifiés pour détecter un front descendant plutôt qu'un front montant.

Sur la figure suivante, plusieurs éléments peuvent être observés :

- Détection du front descendant : le signal Detectar passe à '1' uniquement lorsque KEY0 effectue une transition de '1' vers '0'.
- Comportement séquentiel : Detectar ne change pas d'état instantanément lors de la transition de KEY0, mais uniquement au front montant de l'horloge (clk) suivant, ce qui correspond au comportement attendu d'un circuit synchrone utilisant `rising_edge(clk)`.
- Durée de l'impulsion : l'impulsion sur Detectar dure exactement une période d'horloge (10 ps), puis revient à '0' car la condition de détection n'est plus satisfaite au cycle suivant.
- Initialisation (Reset) : au début de la simulation, lorsque `rst_n` est à '0', tous les signaux sont correctement initialisés, évitant ainsi l'apparition de valeurs indéfinies.
- Sur la simulation présentée :
 - Premier front descendant : KEY0 passe à '0' vers 35 ps ; au front d'horloge suivant, à 45 ps, le signal Detectar passe à '1'.
 - Second front descendant : KEY0 passe à '0' vers 75 ps ; au front d'horloge de 85 ps, Detectar est de nouveau activé.

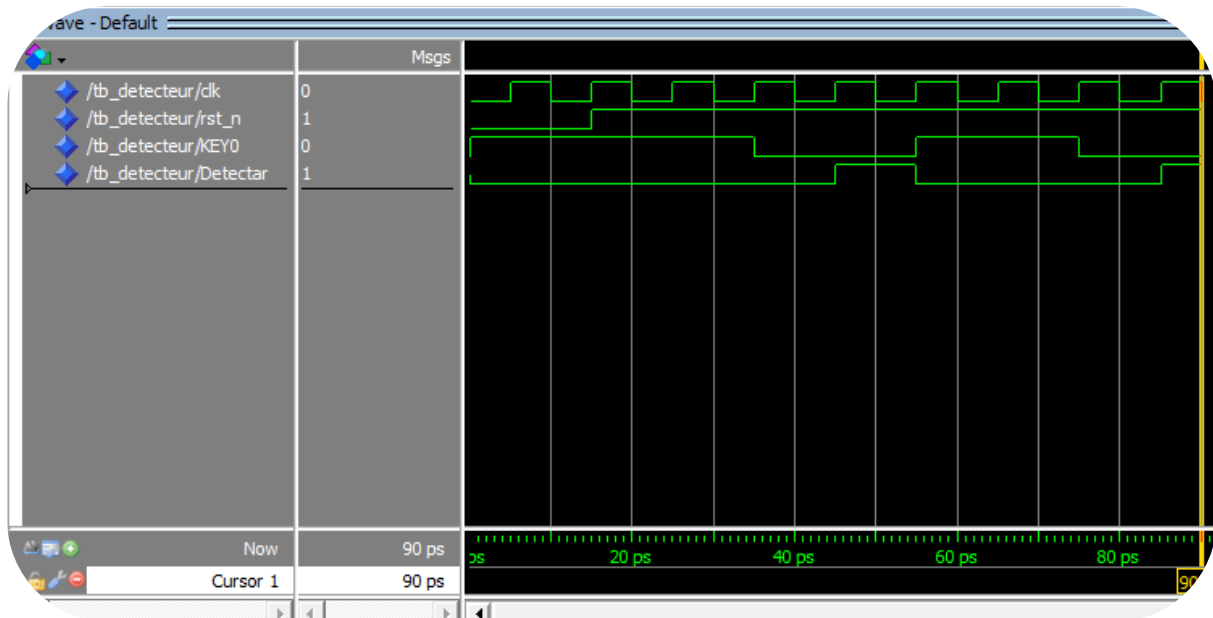


Figure 15 : Simulation détecteur de front

III. Quartus prime

Pour la partie réalisée avec Quartus Prime, j'ai choisi de ne pas nommer le projet *serrure_electronique* mais *top_module*. Ce choix a été fait afin de rester cohérent avec la base de travail déjà effectué sous ModelSim. L'ensemble des simulations fonctionnait correctement, et par précaution, j'ai préféré ne pas modifier le nom du fichier ni celui de l'entité VHDL, afin d'éviter tout risque de dysfonctionnement. Le projet Quartus porte donc le nom *top_module* et est organisé dans un dossier dédié nommé *Quartus*.

Dans cette partie du projet, je me suis appuyé sur la documentation et la CAO électronique de la carte DE10-Lite afin d'identifier le raccordement des broches (*pins*). Une difficulté rencontrée concernait l'apparition de broches non souhaitées dans Quartus Prime, le *buscode*. Dans un premier temps, j'ai laissé Quartus affecter automatiquement les broches, en considérant que l'outil pouvait adapter les connexions en fonction de la carte électronique. Cependant, cette configuration automatique ne donnait pas le résultat attendu, voir même ne fonctionnait pas.

J'ai alors choisi de revenir sur mes fichiers VHDL dans l'environnement ModelSim et VScode afin d'analyser, corriger et valider le fonctionnement du code par simulation. Une fois le comportement confirmé à priori, j'ai copié les fichiers VHDL validés dans le dossier du projet Quartus. Après cette mise à jour, les broches problématiques sont apparues comme *Unknown*, ce qui indique normalement qu'elles sont désormais gérées en interne et qu'il n'est plus nécessaire de les affecter.



All Pins											
Named: *											Filter: Pins: all
Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation
Busint[3]	Input	PIN_C12	7	B7_NO	PIN_C12	3.3-V LVTTTL		8mA (default)			
Busint[2]	Input	PIN_D12	7	B7_NO	PIN_D12	3.3-V LVTTTL		8mA (default)			
Busint[1]	Input	PIN_C11	7	B7_NO	PIN_C11	3.3-V LVTTTL		8mA (default)			
Busint[0]	Input	PIN_C10	7	B7_NO	PIN_C10	3.3-V LVTTTL		8mA (default)			
clk	Input	PIN_P11	3	B3_NO	PIN_P11	3.3-V LVTTTL		8mA (default)			
HEX0[5]	Output	PIN_C17	7	B7_NO	PIN_C17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[5]	Output	PIN_D17	7	B7_NO	PIN_D17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[4]	Output	PIN_E16	7	B7_NO	PIN_E16	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[3]	Output	PIN_C16	7	B7_NO	PIN_C16	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[2]	Output	PIN_C15	7	B7_NO	PIN_C15	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[1]	Output	PIN_E15	7	B7_NO	PIN_E15	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX0[0]	Output	PIN_C14	7	B7_NO	PIN_C14	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[6]	Output	PIN_B17	7	B7_NO	PIN_B17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[5]	Output	PIN_A18	7	B7_NO	PIN_A18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[4]	Output	PIN_A17	7	B7_NO	PIN_A17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[3]	Output	PIN_B16	7	B7_NO	PIN_B16	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[2]	Output	PIN_E18	6	B6_NO	PIN_E18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[1]	Output	PIN_D18	6	B6_NO	PIN_D18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX1[0]	Output	PIN_C18	7	B7_NO	PIN_C18	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[6]	Output	PIN_B22	6	B6_NO	PIN_B22	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[5]	Output	PIN_C22	6	B6_NO	PIN_C22	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[4]	Output	PIN_B21	6	B6_NO	PIN_B21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[3]	Output	PIN_A21	6	B6_NO	PIN_A21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[2]	Output	PIN_B19	7	B7_NO	PIN_B19	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[1]	Output	PIN_A20	7	B7_NO	PIN_A20	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[0]	Output	PIN_B20	6	B6_NO	PIN_B20	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX2[5](n)	Output					3.3-V LVTTTL		minimum current	2 (default)		
HEX3[6]	Output	PIN_E17	6	B6_NO	PIN_E17	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[5]	Output	PIN_D19	6	B6_NO	PIN_D19	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[4]	Output	PIN_C20	6	B6_NO	PIN_C20	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[3]	Output	PIN_C19	7	B7_NO	PIN_C19	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[2]	Output	PIN_E21	6	B6_NO	PIN_E21	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[1]	Output	PIN_E22	6	B6_NO	PIN_E22	3.3-V LVTTTL		8mA (default)	2 (default)		
HEX3[0]	Output	PIN_F21	6	B6_NO	PIN_F21	3.3-V LVTTTL		8mA (default)	2 (default)		
Ouverture	Output	PIN_A8	7	B7_NO	PIN_A8	3.3-V LVTTTL		8mA (default)	2 (default)		
rst_n	Input	PIN_B8	7	B7_NO	PIN_B8	3.3-V LVTTTL		8mA (default)			
Buscode[4]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[5]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[15]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[13]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[14]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[12]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[11]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[10]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[9]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[8]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[7]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[6]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[3]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[2]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[1]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode[0]	Unknown					3.3-V LVTTTL		8mA (default)			
Buscode	Unknown					3.3-V LVTTTL		8mA (default)			
<<new node>>											

Sources

Lors de ce projet différentes sources ont été utiliser pour ce mini-projet, voici les liens de ceux-ci.

<https://irfu.cea.fr/en/Phoceaf/file.php?class=cours&file=/herve.le-provost/langageVHDL.pdf>

<https://rti.etf.bg.ac.rs/rti/ri5rvl/tutorial/Html/Tuttbch/Tuttbch.htm#:~:text=A%20test%20bench%20is%20a,ports%20of%20the%20UUT%20entity.>

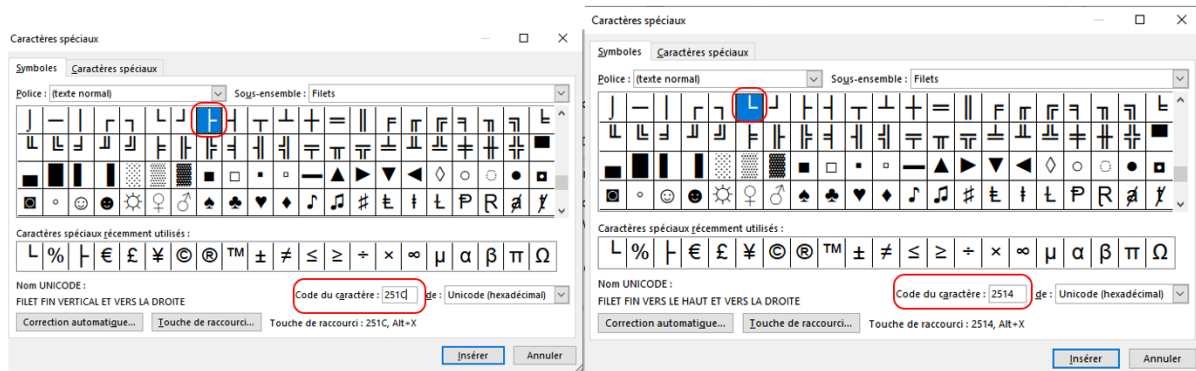
<https://www.eng.auburn.edu/~nelson/courses/elec4200/Slides/VHDL%206%20Testbench.pdf>

<https://hal.science/hal-04301440v1/document>

<https://stackoverflow.com/questions/29267616/vhdl-if-statements-and-process-names>

Annexe

Caractères spéciaux :



Tables des figures

Figure 1 : Carte DE10-Lite	6
Figure 2 : Représentation graphique de l'ALU	8
Figure 3 : Simulation ALU	10
Figure 4 : Représentation graphique du Banc de registre	10
Figure 5 : Simulation Datapath (Banc de registre + ALU)	12
Figure 6 : Représentation graphique du Multiplexeur 2 vers 1	13
Figure 7 : Simulation du Multiplexeur 2 vers 1	14
Figure 8 : Représentation graphique de l'Extension de signe	15
Figure 9 : Simulation de l'Extension du signe	16
Figure 10 : Représentation graphique de la Memory data	17
Figure 11 : Simulation de la Memory Data	19
Figure 12 : Représentation graphique du Datapath	19
Figure 13 : Assemblage des modules créer précédement.....	20
Figure 14 : Simulation du Datapaht	21
Figure 15 : Simulation détecteur de front	25